DONANIM + YAZILIM REFERANS BELGESI

LinoRobot2 Yapım Kılavuzu

ESP32-S3 ve Raspberry Pi 4 üzerine, `linorobot2` / `linorobot2_hardware` açık kaynak ekosistemiyle inşa edilmiş, özel (custom) mekanik parçalara sahip bir diferansiyel sürüşlü mobil robotun sıfırdan yeniden üretilebilmesi için gereken malzeme listesi, güç mimarisi, kablolama şeması, yazılım kurulumu ve kalibrasyon adımlarının tamamı.

TABAN	MCU	BILGISAYAR	LIDAR	BATARYA	YAZILIM
2WD / Diferansiyel	ESP32-S3-DevKitC-1	Raspberry Pi 4	LDS-01 (hls_lfcd_lds_driver)	3S1P 11.1V LiPo	ROS 2 Humble

DEVAM EDEN ARAŞTIRMA

Bu platform, WiFi kapsama alanı haritalaması üzerine yürütülecek bir araştırma projesinin temelini oluşturmaktadır. Burada belgelenen otonom hareket, haritalama (SLAM) ve navigasyon (Nav2) işlevleri bu platformda yer almaktadır; WiFi sinyal ölçümünün konum bilgisiyle eşleştirilmesi, kapsama haritası üretimi ve gerçek sahada doğrulama bu temel üzerine geliştirilecek, ayrı bir çalışma kapsamında ele alınacaktır.

---

§01 Genel Bakış

Bu robot, TurtleBot3 Burger sınıfına yakın ölçülerde ama tamamen özel tasarlanmış bir platformdur: mekanik parçalar (motor oturağı, gövde katmanları, tekerlek ve şaft adaptörü) 3D yazıcıdan basılır; elektronik, firmware ve yazılım tarafı bu kılavuzda anlatılan açık kaynak yığınla kurulur.

Sistem mimarisi — üç katman

Kontrol üç ayrı katmana bölünmüştür, her biri farklı bir gerçek-zamanlılık ihtiyacını karşılar:

KATMAN	GÖREV	NEDEN AYRI
ESP32-S3 (firmware)	20ms/50Hz döngüyle motor PID, encoder okuma, IMU/odometri yayını	Gerçek zamanlı, kesintisiz döngü gerektirir — bir işletim sistemi üzerinde garanti edilemez
Raspberry Pi 4 (robot bilgisayarı)	micro-ROS agent, LIDAR sürücüsü, EKF, (opsiyonel) SLAM/Nav2	ROS 2 çalıştırabilecek güçte ama robot üzerinde taşınabilir, düşük güçlü
PC / Workstation (opsiyonel)	RViz görselleştirme, ağır Nav2/SLAM hesaplaması	RPi4'ün CPU sınırını aşan görevler için — bkz. §10

TASARIM FELSEFESI

Her teknik karar mümkün olduğunca resmi datasheet veya kaynak koda dayandırılmalı, tahmine dayanmamalıdır. Bu kılavuzdaki her voltaj/akım değeri ilgili üreticinin (Pololu, Slamtec, Infineon/BTS7960 modül üreticisi, Raspberry Pi Vakfı) resmi belgelerinden alınmıştır.

---

§02 Malzeme Listesi

Elektronik ve hesaplama bileşenleri hazır/satın alınan parçalardır; mekanik gövde ise özel tasarımdır (bu kılavuzda genel ilkeleri anlatılır, CAD dosyası kapsam dışıdır).

HESAPLAMA & KONTROL

BILEŞEN	MODEL	ADET	NOT
Mikrodenetleyici	ESP32-S3-DevKitC-1 (WROOM-1, N8R2)	1	8MB flash / 2MB PSRAM, dahili native USB — harici USB-TTL adaptörü gerekmez
Robot bilgisayarı	Raspberry Pi 4 Model B, 4GB	1	Ubuntu Server 22.04.5 LTS 64-bit (Desktop değil — RAM tasarrufu)
microSD kart	≥32GB, Class 10 / A2	1	—

TAHRIK (MOTOR & SÜRÜCÜ)

BILEŞEN	MODEL	ADET	NOT
Redüktörlü DC motor	Pololu 25D, 99:1, #3219	2	12V, 100 RPM (yüksüz), dahili 48 CPR Hall encoder, 4mm D-şaft
Motor sürücüsü	BTS7960 (IBT_2 modül kartı)	2	Motor başına 1 sürücü — H-köprü, PWM kontrollü, 6-27V aralığı
Tekerlek + şaft adaptörü	Özel tasarım (3D baskı)	2 + 2	Bkz. §05 — şaft oturma toleransı kritik

ALGILAMA

BILEŞEN	MODEL	ADET	NOT
2D LIDAR	LDS-01 (ROBOTIS)	1	CP2102 USB-UART adaptörlü — tek USB kablosu yeterli (veri+güç).

GÜÇ

BILEŞEN	MODEL	ADET	NOT
Batarya	3S1P LiPo, 11.1V, ≥1800mAh, ≥5C	1	C değeri ne kadar yüksekse o kadar iyi — bkz. §03 risk notu
Step-down (buck) regülatör	XL4015E1 modül, ~5A	1	12V(11.1V) → 5V, tüm lojik/bilgisayar hattının tek kaynağı
Bağlantı	Batarya konnektörü, USB Type-C kablo (kesilecek)	1+1	Type-C'den sadece Kırmızı(+5V)/Siyah(GND) lehimlenir

MEKANIK & SARF

BILEŞEN	NOT
Gövde katmanları, motor oturağı, sürücü braketleri	3D baskı, özel tasarım — bkz. §05
Distans (spacer) çubuklar + M3 somun	Katmanlar arası montaj için
M3 vida/somun seti, 1.5mm Allen anahtar	Genel montaj + şaft sabitleme
Multimetre	Alet — voltaj/akım doğrulaması için zorunlu, bkz. §09

§03 Güç Mimarisi

Tek bir batarya, tek bir regülatör (XL4015E1) üzerinden tüm sistemi besler. Motorlar ham (regüle edilmemiş) batarya gerilimini alır; RPi4 ve sürücü lojik hatları regüle 5V'u paylaşır; encoderlar ESP32-S3'ün kendi 3.3V pininden beslenir (5V'tan **asla** değil — GPIO'lar 3.3V'a dayanıklıdır).

Güç bütçesi (üretici datasheet'lerinden)

BILEŞEN	GERILIM	TIPIK AKIM	AZAMI / AÇILIŞ AKIMI
Raspberry Pi 4	5V (USB-C)	0.21–0.65A (ölçülen, boşta–tam yük)	3A (resmi giriş özelliği)
ESP32-S3-DevKitC-1	5V (RPi4 USB üzerinden)	~0.25A	–
LDS-01 (ROBOTIS resmi e-manual)	5V ±5%	≤400mA	~1A (rush)
Pololu 25D motor (×2)	11.1V (ham)	~150–300mA / motor (normal sürüş)	~5.6A / motor (kilitlenme/stall)
BTS7960 sürücü (×2)	5V lojik + 11.1V güç	~50mA lojik / sürücü	Modül reyting: ~43A/kanal (kısa süreli, kullanılmıyor)
Toplam (bataryaya yansıyan, en kötü durum)	**11.1V eşdeğer**	**~12.6A (iki motor aynı anda kilitlenirse)**	

BILINEN RISK

5C'lik bir bataryanın (ör. 1800mAh × 5C = 9A azami) sürekli akım kapasitesi, iki motorun aynı anda kilitlenmesi (engele çarpma) senaryosundaki ~11.2A ihtiyacın **altında** kalır. Firmware'de akım sınırlama yoktur (`BATTERY_PIN` / `USE_INA219` tanımlı ama kullanılmaz — bkz. §10). Mümkünse ≥10C, tercihen ≥20C bir batarya seçilmesi önerilir; düşük C'li bir batarya kullanılacaksa risk bilinçli olarak kabul edilmelidir.

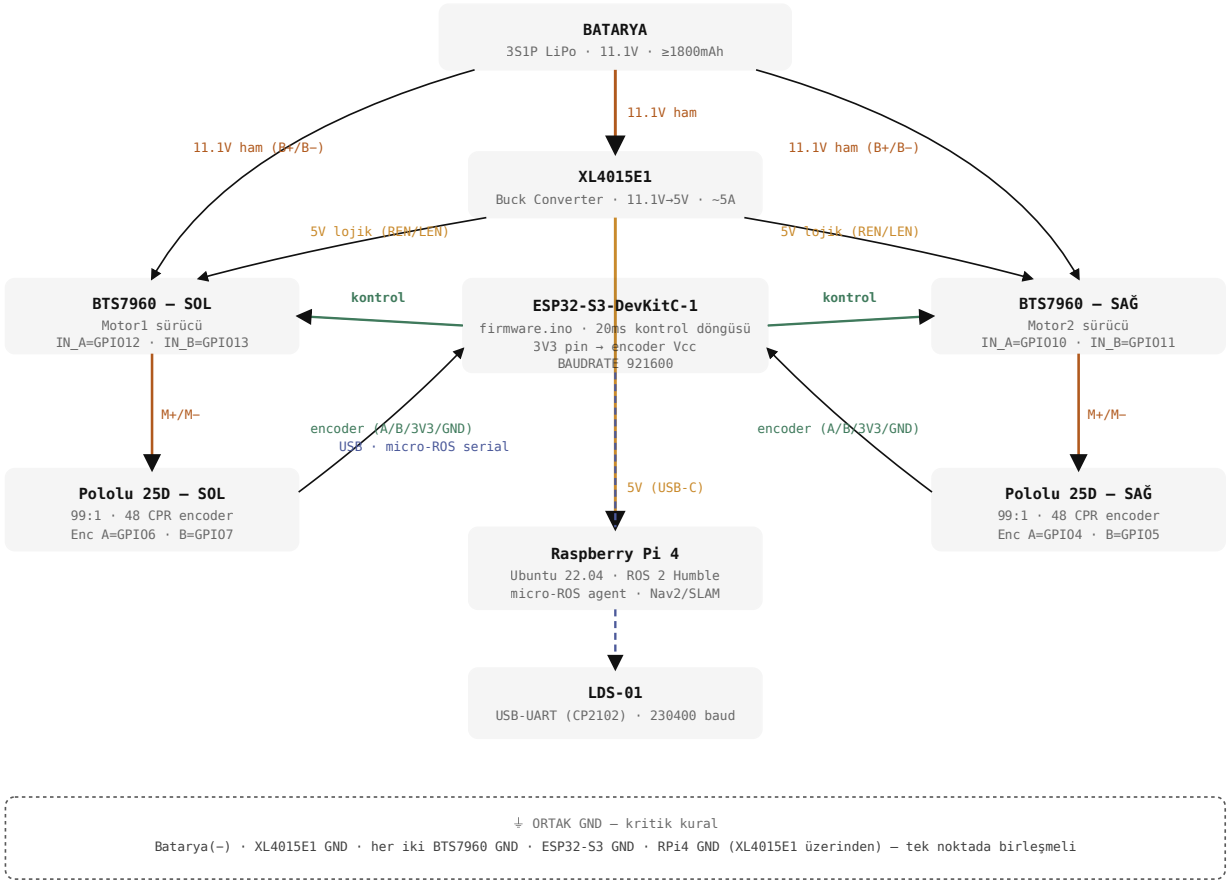
Güç dağıtım noktası — XL4015E1

XL4015E1'in giriş ve çıkış tarafı her biri ikiye ayrılır:

TARAF	KAYNAK/HEDEF	GERILIM
Giriş	Batarya (+/–)	11.1V ham
	BTS7960 ×2, B+/B– (motor güç girişi)	11.1V ham
Çıkış (regüle)	Raspberry Pi 4, USB-C güç girişi	5V
	BTS7960 ×2, REN/LEN + VCC (lojik/etkinleştirme)	5V

§04 Kablolama Şeması

Aşağıdaki şema, güç hatlarını (renk kodlu) ve sinyal/veri bağlantılarını tek diyagramda gösterir. Tüm pin numaraları `esp32s3_config.h` kaynağından doğrudan alınmıştır.



Güç, batarya(11.1V)'dan hem doğrudan iki BTS7960 sürücüsüne hem de XL4015E1 üzerinden 5V'a düşürülüp RPi4 ve sürücü lojik hatlarına dağılır. ESP32-S3, her iki sürücüyü kontrol sinyaliyle yönetir ve her iki motorun encoderini okur; RPi4 ile arasında ayrı bir USB veri hattı (micro-ROS) vardır.

Tam pin haritası

İŞLEV	ESP32-S3 PINI	KAYNAK
Motor1 (SOL) sürücü IN_A / IN_B	`GPIO12` / `GPIO13`	`esp32s3_config.h`
Motor2 (SAĞ) sürücü IN_A / IN_B	`GPIO10` / `GPIO11`	`esp32s3_config.h`
Motor1 (SOL) encoder A / B	`GPIO6` / `GPIO7`	`esp32s3_config.h`
Motor2 (SAĞ) encoder A / B	`GPIO4` / `GPIO5`	`esp32s3_config.h`
Encoder Vcc (her ikisi)	`3V3` (GPIO değil, güç pini)	Board üzerindeki 3.3V çıkışı — 5V'tan asla beslenmez
I2C (gelecekteki IMU için, opsiyonel)	`SDA=GPIO8` / `SCL=GPIO9`	`esp32s3_config.h`

POLOLU 6 TELLI DEMET RENK KODU

Kırmızı=M+ · Siyah=M− · Sarı=Enc A · Beyaz=Enc B · Mavi=Enc Vcc(3.3V) · Yeşil=Enc GND

Yukarıdaki şemanın güç/sinyal/ortak-toprak ayrımını üç ayrı figürde ve bir güç-verilmeden-önce kontrol listesiyle genişleten referans: `kablo_semasi.html`.

---

§05 Mekanik Montaj

Gövde, tek seferde değil **alttan yukarıya doğru katman katman** monte edilir; her katman bir öncekinin üzerine sarı spacer çubuklar ve somunlarla oturur.

1. **Katman 1 — Motorlar & sürücüler**

   İki Pololu motor ve iki BTS7960 sürücü en alt katmana sabitlenir. Motorların hangisinin SOL/SAĞ olacağı burada fiziksel olarak belirlenir — pin haritasıyla (§04) tutarlı olmalı.

2. **Katman 2 — ESP32-S3 & XL4015E1**

   Mikrodenetleyici ve güç regülatörü ikinci katmana yerleştirilir; motor/encoder kabloları bu katmana çıkar.

3. **Katman 3 — Raspberry Pi 4**

   Üçüncü katman RPi4'ü taşır; XL4015E1'in 5V çıkışı buraya USB-C ile bağlanır.

4. **Katman 4 — LIDAR**

   En üst katman LIDAR'ı taşır — 360° görüş açısı için diğer katmanların üzerinde, engelsiz bir konumda olmalı.

5. **Tekerlek montajı**

   Motor D-şaftına önce şaft adaptörü, sonra tekerlek geçirilir. **Bu arayüz, projenin en kritik tolerans noktasıdır** — aşağıdaki tasarım notuna dikkat edin.

TASARIM NOTU — ŞAFT ADAPTÖRÜ TOLERANSI

Motor şaftı 4mm D-profil, plastik (3D baskı) bir adaptöre *zorlanarak* geçirilirse, plastik mikro düzeyde deforme olup merkezden kayar — bu, tekerlek dönerken gözle görülür bir sallanmaya ("8 çizme") yol açar. Doğru yaklaşım: deliği şafttan hafif büyük (4.0–4.2mm) tasarlayıp hiç zorlamadan geçirmek, sabitlemeyi bir **set vidası + M3 somun cebi** (şaftın düz D-yüzeyine bastıran) ile yapmak, delik çevresinde en az 3–4mm dolu et kalınlığı bırakmak, ve baskı sonrası deliği gerçek bir 4mm matkap ucuyla raybalamak.

---

§06 ESP32-S3 Firmware Kurulumu

Yükleme yöntemi PlatformIO'dur (Arduino IDE değil) — micro-ROS yalnızca PlatformIO ile derlenebilir. ESP32-S3'ün dahili native USB'si sayesinde ayrı bir USB-TTL adaptörüne gerek yoktur.

1. **PlatformIO CLI kurulumu**

```
pip install platformio
```

2. **linorobot2_hardware deposunu klonla**

```
git clone -b humble https://github.com/linorobot/linorobot2_hardware.git
cd linorobot2_hardware/firmware
```

3. **Doğru config'i doğrula**

`config.h` içinde `USE_ESP32S3_CONFIG` tanımlı olmalı; motor/pin/kalibrasyon değerleri `config/custom/esp32s3_config.h` içindedir (§04'teki pin haritasıyla aynı).

4. **İki zorunlu kod yaması (upstream'den fark)**

Bu robotta, resmi depodan sadece **iki dosyada, iki satır** değiştirildi — bunlar olmadan motorlar hiç dönmez / encoder hiç veri üretmez. Aşağıdaki iki değişikliği uygulayın.

5. **Derle ve yükle**

```
pio run -e esp32s3 -t upload --upload-port /dev/ttyACM0
```

Native USB portunda otomatik reset devresi olmadığından, yükleme sırasında **BOOT** tuşuna basılı tutup **RESET**'e dokunmak gerekebilir.

6. **Doğru portu ayırt et**

Kartın iki USB-C portu vardır: `idVendor=303a` (native USB — Serial/micro-ROS için kullanılan budur) ve `idVendor=1a86` (UART köprü — sadece flaşlama). Karıştırmayın.

Yama 1 — `firmware/lib/encoder/encoder.h` (satır 52)

Açık-kolektör (open-collector) encoder'lar için `ESP32Encoder` kütüphanesinin varsayılan pull-DOWN ayarı hiç veri üretmez; pull-UP'a zorlanmalı. `attachHalfQuad()` çağrısından **hemen önce**, `Encoder` yapıcısının (constructor) içine eklenir:

```
Encoder(int pin1, int pin2, int counts_per_rev, bool invert = false) {
    int temp_pin = pin1;
    if (pin1 < 0 || pin2 < 0) return; // unused encoder
    if(invert)
    {
        pin1 = pin2;
        pin2 = temp_pin;
    }
    counts_per_rev_ = counts_per_rev;
    ESP32Encoder::useInternalWeakPullResistors = puType::up;  // <-- EKLENEN SATIR
    encoder_.attachHalfQuad(pin1, pin2);
}
```

Yama 2 — `firmware/src/firmware.ino` (satır ~295)

ROS2 Humble'ın `rclc` kütüphanesinde `rclc_timer_init_default2` (5 argümanlı) yoktur, sadece 4 argümanlı `rclc_timer_init_default` vardır. Upstream kod bazı

dallarda 5 argümanlı ismi kullanıyor olabilir — Humble'da derleme hatası verir,
aşağıdaki gibi olmalıdır:

```
// create timer for actuating the motors at 50 Hz (1000/20)
const unsigned int control_timeout = 20;
RCCHECK(rclc_timer_init_default(   // <-- "rclc_timer_init_default2" DEĞİL
    &control_timer,
    &support,
    RCL_MS_TO_NS(control_timeout),
    controlCallback
));
```

---

§07 Raspberry Pi 4 Kurulumu — Komut Günlüğü

Bu bölüm, RPi4 üzerinde fiilen çalıştırılan **tüm komutları**, sırasıyla, hiçbir adım atlanmadan belgeler. Amaç: aynı kartı sıfırdan, bu kılavuzu takip ederek, hiçbir komutu tahmin etmeden yeniden kurabilmek.

Faz 1 — İşletim sistemi

1. **Raspberry Pi Imager ile yazma**

   Seçilen imaj: **Ubuntu Server 22.04.5 LTS (64-bit)** — Desktop değil (gereksiz RAM/CPU), 32-bit değil (ROS2 Humble'ın resmi `packages.ros.org` deposu sadece arm64 sağlıyor).

   Imager'ın gelişmiş ayarları (⚙️) ile SD karta yazmadan önce yapılandırılanlar: Hostname `raspberrypi4`, SSH etkin (şifre ile), kullanıcı `pi4`, WiFi SSID/şifre, saat dilimi `Europe/Istanbul`, klavye düzeni `tr`.

2. **İlk SSH bağlantısı**

   ```
   ssh pi4@<rpi-ip>
   ```

   IP, DHCP ile atanır; her açılışta değişebilir (router'ın DHCP istemci listesinden ya da `ping` ile ARP taramasından bulunabilir — bkz. §10).

   ```
   uname -m   # aarch64 çıkmalı — 64-bit doğrulaması
   ```

3. **Sistem güncelleme**

   ```
   sudo apt update && sudo apt upgrade -y
   ```

   Karşılaşılan sorun: `Hash Sum mismatch` hatası (birkaç farklı pakette, rastgele). Geçici çözüm: komutu tekrar çalıştırmak (genelde 2. denemede geçiyor). Kalıcı kök neden ve çözüm için bkz. §10 — bu, kurumsal ağın şeffaf HTTP proxy'sinden kaynaklanıyordu.

   Kernel güncellenirse (`5.15.0-1061` → `5.15.0-1105-raspi` gibi) yeniden başlatın: `sudo reboot`

4. **Locale (dil/karakter kodlaması)**

   ```
   locale
   sudo apt install locales -y
   sudo locale-gen en_US en_US.UTF-8
   sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
   export LANG=en_US.UTF-8
   ```

   ROS2'nin DDS katmanı ve `colcon` / `rosdep` UTF-8'e güveniyor; minimal Ubuntu Server kurulumunda locale bazen `C` / `POSIX` kalıyor.

Faz 2 — ROS 2 Humble

1. **Resmi ROS2 apt deposu**

   ```
   sudo apt install software-properties-common -y
   sudo add-apt-repository universe
   sudo apt update && sudo apt install curl -y
   sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ro
   echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] htt
   sudo apt update
   ```

2. **Paket kurulumu (ros-base, desktop değil)**

```
sudo apt install ros-humble-ros-base -y
sudo apt install python3-colcon-common-extensions python3-rosdep python3-argcomplete -y
sudo rosdep init
rosdep update
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

Neden `ros-base`, `desktop` değil: RPi'ye ekran bağlanmıyor (başlangıçta) — RViz/Gazebo gibi GUI araçlarına ihtiyaç yok, gereksiz paket/RAM kullanımından kaçınılıyor. (Not: proje ilerledikçe RPi4'e fiziksel ekran da bağlandı ve GNOME masaüstü sonradan kuruldu — bkz. §10, bu normalde RPi3B+'ta önerilmeyen bir şeydi ama RPi4'ün 4GB RAM'i bunu kaldırabildi.)

Faz 3 — micro-ROS agent workspace

1. **Agent kaynak kodu ve derleme**

```
sudo apt install git -y
mkdir -p ~/microros_ws/src
cd ~/microros_ws
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git src/micro_ros_setup
rosdep install --from-paths src --ignore-src -y
colcon build --symlink-install
source install/local_setup.bash
```

Bu adım sadece `micro_ros_setup` betik paketini derler (birkaç saniye). Asıl agent henüz yok.

2. **Agent'ın kendisini oluştur ve derle**

```
sudo apt install python3-vcstool -y
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
```

RPi4'te (4GB RAM) ~4-5 dakika sürer. Düşük RAM'li kartlarda (RPi3B+ gibi, 1GB) swap dosyası gerekebilir — `/etc/fstab`'a eklenmezse her reboot'ta kaybolur.

3. **Test (ESP32 bağlıyken)**

```
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0 -b 921600
```

Başka bir terminalde `ros2 topic list` ile `/cmd_vel`, `/odom/unfiltered`, `/imu/data` görülmeli.

Faz 4 — Tam linorobot2 kurulumu (Nav2/SLAM/URDF)

1. **Resmi kurulum betiği**

```
source /opt/ros/humble/setup.bash
cd /tmp
wget https://raw.githubusercontent.com/linorobot/linorobot2/humble/install_linorobot2.bash
bash install_linorobot2.bash 2wd -
```

Argümanlar: `2wd` = robot taban tipi, ikinci argüman LIDAR modeli (`ydlidar`, `xv11`, `ld06`, `ld19`, `stl27l`, `a1` / `a2` / `a3` / `c1` / `s1` / `s2` / `s3` — LDS-01 bu listede yok). `-` = "lazer sensör yok", betiğin kendi LIDAR kurulumunu atlar; LDS-01 bir sonraki adımda ayrıca eklenir.

Betik `~/linorobot2_ws`'u oluşturur: URDF (`linorobot2_description`), Nav2 config (`linorobot2_navigation`), SLAM Toolbox config, ikinci bir bağımsız micro-ROS agent kopyası. `LINOROBOT2_BASE=2wd` ortam değişkeni `~/.bashrc`'ye eklenir.

2. **LDS-01 sürücüsünü elle ekle**

```
cd ~/linorobot2_ws/src
git clone -b humble https://github.com/ROBOTIS-GIT/hls_lfcd_lds_driver.git
cd ~/linorobot2_ws
colcon build --packages-select hls_lfcd_lds_driver
source install/setup.bash
```

Çalıştırma: `ros2 launch hls_lfcd_lds_driver hlds_laser.launch.py` (bkz. scripts/start_robot.sh). `linorobot2_bringup.service` 'in `ExecStart` 'ı (systemd/install_agent_service.sh) `LINOROBOT2_LASER_SENSOR` 'ü unset eder, böylece bringup hiçbir LIDAR'ı otomatik başlatmaya çalışmaz — LIDAR her zaman `start_robot.sh` ile ayrıca başlatılır.

3 **Elle bringup (systemd kurulmadan önce, ilk test için)**

```
ros2 launch linorobot2_bringup bringup.launch.py micro_ros_baudrate:=921600
```

`micro_ros_baudrate` mutlaka `esp32s3_config.h` 'deki `BAUDRATE` (921600) ile eşleşmelidir — varsayılan (115200) farklıdır ve agent ESP32 ile hiç konuşamaz.

§08 Kalibrasyon

ROS2'ye hiç bağlanmadan, repo'nun kendi `calibration/` firmware'iyle motor/tekerlek değerleri ölçülür.

PARAMETRE	NASIL ÖLÇÜLÜR	BU PROJEDE ÖLÇÜLEN DEĞER
`MOTOR_MAX_RPM`	Üreticinin datasheet'i	`100 RPM (yüksüz, 12V)`
`COUNTS_PER_REV`	1m/2m mezura testiyle geriye hesaplanan düzeltme (bkz. altındaki not)	`M1(SOL)=2296 · M2(SAĞ)=2327`
`WHEEL_DIAMETER`	Tur testi: tekerleğe işaret koyup 5 tam tur sürdürüp zeminde kat edilen mesafeyi ölçmek — `d_eff = mesafe/(tur×π)`	`0.0646 m (fiziksel kumpas: 0.066 m)`
`LR_WHEELS_DISTANCE`	Sol-sağ tekerlek merkezleri arası — Waffle şasiye geçişte yeniden ölçüldü	`0.30 m (Burger şasi: 0.160 m)`
`K_P / K_I / K_D`	Stok değerlerle başla, sadece gerekirse ayarla	`0.6 / 0.8 / 0.5`

BILINEN FIRMWARE HATASI (UPSTREAM)

Kalibrasyon firmware'inin `sampleMotors()` fonksiyonu, `spin()` çağrısını bir `while(true)` döngüsü içinde saniyede yüz binlerce kez tekrarlarsa, ESP32'nin PWM donanımı (LEDC) geçerli bir sinyal üretemez ve motor **hiç dönmez**. Çözüm: `spin()` 'i döngünün *dışına, öncesine* taşımak — döngü sadece zamanlamayı kontrol etmeli, PWM komutunu tekrar tekrar göndermemelidir.

Düzeltilmiş `sampleMotors()` — `calibration/src/firmware.ino`

Bu robotta fiilen kullanılan, düzeltilmiş hâli (`spin()` döngünün dışında, bir kere):

```
void sampleMotors(bool show_summary)
{
    if(Kinematics::LINO_BASE == Kinematics::DIFFERENTIAL_DRIVE)
    {
        total_motors = 2;
    }

    float measured_voltage = constrain(MOTOR_POWER_MEASURED_VOLTAGE, 0, MOTOR_OPERATING_VOLTAGE);
    float scaled_max_rpm = ((measured_voltage / MOTOR_OPERATING_VOLTAGE) * MOTOR_MAX_RPM);
    float total_rev = scaled_max_rpm * (SAMPLE_TIME / 60.0);

    for(int i=0; i<total_motors; i++)
    {
        Serial.print("SPINNING ");
        Serial.print(labels[i]);

        unsigned long start_time = micros();
        unsigned long last_status = micros();

        encoders[i]->write(0);
        motors[i]->spin(PWM_MAX);          // <-- DÖNGÜ DIŞINDA, TEK SEFER
        while(true)
        {
            if(micros() - start_time >= SAMPLE_TIME * 1000000)
            {
                motors[i]->spin(0);
                Serial.println("");
                break;
            }

            if(micros() - last_status >= 1000000)
            {
                last_status = micros();
                Serial.print(".");
            }
        }

        counts_per_rev[i] = encoders[i]->read() / total_rev;
    }
    if(show_summary)
        printSummary();
}
```

BU KOMUTUN SAYILARINA KÖRÜ KÖRÜNE GÜVENMEYIN

`sample` 'ın ürettiği `COUNTS_PER_REV` , `MOTOR_MAX_RPM` 'in gerçek devirle birebir uyuştuğu varsayımına dayanır — bu projede bu varsayım güvenilmez çıktı (bkz. aşağıdaki bölüm). Kullanım komutu: seri port 9600 baud'da `sample` yazıp Enter'a basmak, iki motor için ~10'ar saniye sürer.

`COUNTS_PER_REV` — mezura ile geriye hesaplama yöntemi

Kalibrasyon firmware'inin `sample` komutu `MOTOR_MAX_RPM`'in doğru olduğu varsayımına dayanır (gerçek devri değil, teorik devri kullanır) — bu projede bu varsayım güvenilmez çıktı, iki ayrı denemede birbirinden çok farklı/tutarsız sonuçlar verdi. Onun yerine **odometri ile gerçek mezura ölçümünü karşılaştırıp geriye doğru ölçek hesaplandı**:

1. **1 metre ve 2 metre düz sürüş testi**

   Robot sabit hızda (`/odom/unfiltered` Euclid mesafesi 1.000m / 2.000m'ye ulaşınca duracak şekilde) sürülür, gerçek kat edilen mesafe mezurayla ölçülür.

2. **Ölçek oranı**

   `oran = odometri_mesafesi / gerçek_mesafe` — bu projede 1.001/0.92 ≈ **1.088** (odometri %8.8 fazla okuyordu).

3. **Her iki COUNTS_PER_REV'i aynı oranda büyüt**

   Eski ortalama değer × 1.088 — bu *genel ölçek* hatasını düzeltir (ileri/geri mesafe artık mezura ile birebir tutuyor), ama sol-sağ dengesizliğini (yön sapmasını) düzeltmez.

4. **Yön sapmasını ayrı hesapla**

   2m düz giderken ölçülen yanal sapmadan (bu projede 4.5cm sola) dönüş yarıçapı `R = mesafe² / (2 × sapma)` ve gereken teker hız farkı `L/R` (L = teker arası mesafe) hesaplanır; bu oran, ortalama ölçeği koruyarak iki `COUNTS_PER_REV` arasında yeniden dağıtılır.

> TUZAK — `SAMPLE` KOMUTUNUN ORANINA GÜVENMEYİN
>
> İlk denemede sol-sağ dengesizliği `sample` komutunun ham oranıyla (2287/2233) düzeltilmeye çalışıldı — sonuç **yanlış yönde ve aşırı büyük** bir düzeltme oldu, sapmayı düzeltmek yerine kötüleştirdi. Gerçek mezura ölçümünden geriye hesaplanan oran (yukarıdaki yöntem) doğru sonucu verdi. `sample`'ın mutlak sayıları `MOTOR_MAX_RPM` varsayımına bağlı olduğu için güvenilmez; sadece gerçek dünya ölçümünü referans alın.

---

§09 İlk Çalıştırma ve Doğrulama

Güç vermeden önce ve her aşamada, sırayla doğrulanması gereken kontrol listesi:

- [ ] Multimetre ile XL4015E1 çıkışını RPi4'e bağlamadan **önce** ölç — 5.0–5.2V aralığında olmalı
- [ ] Tüm GND'lerin (batarya, regülatör, her iki sürücü, ESP32-S3) tek noktada ortak olduğunu süreklilik testiyle doğrula
- [ ] Encoder Vcc'nin 3.3V'a bağlı olduğunu, **ESP32'nin RST pinine değil**, teyit et (bu iki pin çoğu kartta yan yanadır — kolay karışır)
- [ ] Motor sürücüsünü güçlendirmeden **başlama** — güçsüz sürücü, seri hatta elektriksel gürültü bindirip ESP32↔RPi4 micro-ROS testine agent'ın çökmesine yol açabilir
- [ ] `ros2 topic echo /odom/unfiltered` ile canlı encoder verisini doğrula
- [ ] Teleop ile dört yönü de (ileri/geri/sol/sağ) tek tek test et, motor kimliğinin (SOL=Motor1/SAĞ=Motor2) doğru olduğunu gözle onayla
- [ ] LIDAR'ı tek başına başlatıp (`ros2 launch hls_lfcd_lds_driver hlds_laser.launch.py`) `ros2 topic hz /scan` ile ~5Hz veri aktığını gör
- [ ] Tam `bringup.launch.py` + `slam.launch.py` ile RViz'de canlı harita oluşumunu izle

---

§10 Bilinen Sorunlar ve Dersler

Bu projenin geliştirilmesi sırasında karşılaşılan ve kök nedeni bulunan sorunlar — sıfırdan aynı yığını kuran biri için en değerli kısım muhtemelen burasıdır.

ELEKTRIKSEL

Ortak GND unutulursa

Belirli yön komutları çalışırken diğerleri "tekler", akım 0A okunur. Her fiziksel yeniden kablolamadan sonra ortak GND'yi elle kontrol edin.

FIRMWARE

Encoder pull-down varsayılanı

`ESP32Encoder` kütüphanesinin varsayılanı pull-DOWN'dır; açık-kolektör encoderlar bununla hiç veri üretmez. `useInternalWeakPullResistors = puType::up;` şart.

FIRMWARE

PWM tight-loop hatası

`spin()` 'in bir döngü içinde tekrar tekrar çağrılması LEDC donanımını bozar. Bkz. §08.

KABLOLAMA

RST / 3.3V pin karışıklığı

Bu iki pin çoğu ESP32 kartında yan yanadır. Encoder Vcc'nin RST'ye bağlanması, kartın kendini sürekli resetlemesine ve gürültülü encoder verisine yol açar.

KINEMATIK

Motor1/Motor2 kimlik takası

Yalnızca düz gitmek doğru ama dönüşler ters çalışıyorsa, tek bir motorun `_INV` bayrağını değil, **Motor1 ↔ Motor2'nin tüm config bloğunu** (pin, encoder, INV) birlikte takas edin.

PERFORMANS

RPi4 + RViz üzerinden WiFi

RViz'i WiFi üzerinden sürekli açık bırakmak, ROS2/DDS'in geri basınç (back-pressure) yaşamasına ve `micro_ros_agent` 'ın senkron kaybetmesine yol açabilir. Sadece kontrol için aç, sonra kapat.

MEKANIK

Şaft-adaptör toleransı

Bkz. §05 — zorlanarak takılan plastik adaptör tekerlek sallanmasının en sık nedenidir.

GÜÇ

Sürücü güçsüzken test etmeyin

Sadece ESP32+RPi4 test edilirken BTS7960 güçsüz bırakılırsa, PWM hattı seri iletişime gürültü bindirip `NotEnoughMemoryException` ile agent'ı çökertebilir.

WIFI

brcmfmac sürücüsü motor yükünde düşüyor

Motor aktivitesinden hemen sonra RPi4'ün WiFi'si tamamen kayboluyor ("No wifi adaptor found"). `lsmod | grep brcm` boş çıkar. Çözüm: `sudo modprobe brcmfmac` — reboot gerekmez.

SLAM/NAV2

LIDAR/SLAM durunca köprü (EKF) kopuyor

LIDAR ya da SLAM'i Ctrl+C ile durdurunca `/odom` (EKF çıktısı) ve TF'teki `odom` çerçevesi kayboluyor — USB kopması yok, EKF düğümü sessizce takılı kalıyor. `/odom/unfiltered` (ham ESP32 verisi) etkilenmiyor, bu yüzden fark edilmesi zor. Çözüm: `sudo systemctl restart linorobot2_bringup.service` . `start_robot.sh` 'nin `cleanup()` fonksiyonu bunu otomatik yapar (bkz. §14).

RVIZ/QOS

LIDAR Best Effort, RViz varsayılanı Reliable

RViz'de `/scan` hiç görünmez, log'da `incompatible QoS` uyarısı çıkar. LIDAR sürücüsü Best Effort yayınlıyor, RViz display'in varsayılanı Reliable. Çözüm: RViz config dosyasında (`.rviz`) LaserScan display'in `Reliability Policy` 'sini `Best Effort` yapın.

GELIŞTIRME ORTAMI

VSCode snap sandbox, GUI uygulamalarını bozuyor

VSCode kendisi snap paketiyse, içindeki terminalden (ya da ona bağlı bir araçtan) `rviz2` gibi GUI uygulamaları `libpthread.so.0 ... undefined symbol` hatasıyla çöker. Çözüm: GUI araçlarını VSCode'un dışındaki bağımsız bir terminalden açın.

AĞ

RPi4'ü IP değiştiğinde MAC'ten bulma

DHCP her açılışta farklı IP verebiliyor. Çözüm: bilinen WiFi MAC'inden (`ip neigh show | grep <mac>` , önce ağı taramak için `ping` ile ARP tablosunu doldurun) cihazı bulun — IP'yi tahmin etmekten çok daha güvenilir.

AĞ

GNOME "No Wi-Fi Adapter Found" ama wlan0 var

Netplan `renderer: networkd` olduğunda, GNOME'un WiFi paneli (NetworkManager'a bakar) cihazı görmez — ama `ip link show wlan0` 'da arayüz gerçekten var. Çözüm: netplan dosyasında `renderer: NetworkManager` yapıp `sudo netplan apply` .

URDF

Encoder/tik kalibrasyonu, URDF ölçüleriyle karışmasın

`esp32s3_config.h` 'deki `WHEEL_DIAMETER` / `COUNTS_PER_REV` odometri hesaplamasını, `2wd_properties.urdf.xacro` 'daki ölçüler ise sadece RViz görselleştirmesini etkiler — ikisi birbirinden bağımsız dosyalardır, biri güncellenince diğeri otomatik güncellenmez.

---

§11 Joystick Teleop Kurulumu

Klavye teleop'a (`teleop_twist_keyboard`) alternatif olarak, Logitech F710 kablosuz gamepad ile D-pad + sağ çubuk kullanılarak sürüş kuruldu.

1 **Paket kurulumu**

```
sudo apt install ros-humble-joy -y
```

Standart `ros-humble-teleop-twist-joy` paketi de denendi, ama bu paket D-pad'i (buton olarak gelen) hız komutuna çeviremediği için (sadece analog çubuk eksenlerini okuyabiliyor) yerine özel bir Python düğümü (`dpad_teleop.py`) yazıldı.

2 **Kumandayı D moduna al**

F710'un arkasındaki D/X anahtarı **D** (DirectInput) konumunda olmalı — Linux'un `joydev` arayüzüyle X moduna göre daha güvenilir çalışıyor.

3 **Ham veriyi test et**

```
ros2 run joy joy_node
# ikinci terminalde:
ros2 topic echo /joy
```

Çubuk/tuş hareketleriyle hangi `axes[]` / `buttons[]` index'inin değiştiğini bulun — bu kontrolcüye özel haritayı aşağıdaki tabloda doğruladık, genel F710 dokümanlarıyla birebir aynı çıkmayabilir.

KONTROL	INDEX	TIP
Sol çubuk sağ/sol — sol çubuk ileri/geri	`axes[0] / axes[1]`	analog
LT / RT (tetikler)	`axes[2] / axes[3]`	analog, boşta 1.0
Sağ çubuk sağ/sol · ileri/geri	`axes[4] / axes[5]`	analog
D-pad sol · aşağı · sağ · yukarı	`buttons[0–3]`	dijital
LB (ölü-adam anahtarı) · RB	`buttons[4] / buttons[5]`	dijital
X · A · B · Y	`buttons[0,1,2,3] (XInput sırası)`	dijital

GÜVENLIK — ÖLÜ ADAM ANAHTARI

`dpad_teleop.py` , **LB (buton 4) basılı tutulmadığı sürece hiçbir hız komutu yayınlamaz**. Bu, yanlışlıkla D-pad'e dokunup robotun fırlamasını önler. Yeni bir kontrolcü kullanacaksanız bu davranışı koruyun.

---

§12 Sistemin Otomatik Başlatılması (systemd)

RPi4 her açıldığında temel sistem (micro-ROS köprüsü + EKF) ve joystick zinciri elle hiçbir şey çalıştırmadan ayağa kalkar. Üç servis, aynı desende kuruldu.

SERVIS	GÖREV	BAĞIMLILIK
`linorobot2_bringup.service`	micro-ROS agent + EKF + joint_state/robot_state publisher	—
`joy_node.service`	F710'u okuyup `/joy` yayınlar	—
`joystick_teleop.service`	`dpad_teleop.py` — `/joy` → `/cmd_vel`	`joy_node.service`, `linorobot2_bringup.service`

Her üçü de `/etc/systemd/system/` altında, tam içerikleri systemd/ klasöründe.
Kurulum:

```
sudo cp systemd/*.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now linorobot2_bringup.service joy_node.service joystick_teleop.service
```

BILINEN KALICI SORUN — HENÜZ TAM ÇÖZÜLMEDI

LIDAR/SLAM durdurulduğunda `linorobot2_bringup.service` 'in içindeki EKF düğümü bazen sessizce takılıp `/odom` ve TF'teki `odom` çerçevesini kaybediyor — servis `active` görünse bile. `start_robot.sh` (§13) bunu her çalıştırmada otomatik yeniden başlatarak aşıyor, ama kök neden (neden koptuğu) tam netleşmedi.

---

§13 SLAM / Nav2 Kullanımı

1 **Haritalama oturumunu başlat**

```
~/start_robot.sh
```

Script (scripts/start_robot.sh): temel sistemin hazır olmasını bekler, LIDAR'ı (`hls_lfcd_lds_driver`) ve `linorobot2_navigation` 'ın `slam.launch.py` 'sini (SLAM Toolbox + tam Nav2 yığınını birlikte) başlatır.

2 **RViz — ayrı bir bilgisayardan**

```
source /opt/ros/humble/setup.bash
source ~/linorobot2_ws/install/setup.bash
ros2 launch linorobot2_viz slam.launch.py
```

Bu, `linorobot2_viz` paketinin kendi `linorobot2_slam.rviz` config'ini açar. **LIDAR verisi görünmüyorsa** §10'daki QoS notuna bakın — Best Effort/Reliable uyumsuzluğu RViz'de sessizce "hiçbir şey göstermeme" ile sonuçlanır, hata vermez.

3 **Durdurma**

Script çalışan terminalde **Ctrl+C** — `cleanup()` fonksiyonu LIDAR/SLAM'i kapatır ve köprüyü otomatik yeniden başlatır (bkz. §12 kutusu).

RVIZ'I WIFI ÜZERINDEN SÜREKLI AÇIK BIRAKMAYIN

Bkz. §10 — DDS geri basıncı (back-pressure) EKF/agent'ın zamanlamasını bozabiliyor. Sadece kontrol için açıp kapatın.

Nav2/EKF parametre ayarları — stoktan farklı olan değerler

Upstream `linorobot2` 'a göre bu robotta değiştirilen parametreler (tam dosyalar §14'te):

DOSYA	PARAMETRE	STOK (UPSTREAM)	BU ROBOTTA
`ekf.yaml`	`frequency`	50.0	10.0
`navigation.yaml`	controller_server: `max_vel_x` / `max_speed_xy`	0.4	0.29
	global/local_costmap: `robot_radius`	0.22	0.13
	velocity_smoother: `max_velocity` / `min_velocity` [0]	±0.5	±0.29

NOT

`0.29 m/s` , joystick teleop'taki (`dpad_teleop.py`) `LINEAR_SPEED` ile aynı değer — robotun test edilmiş güvenli azami hızı. `robot_radius` (0.13), şasinin gerçek ~26cm genişliğine göre ayarlı. Listelenmeyen tüm diğer parametreler stok/upstream değerinde.

§14 Ekler — Tam Dosya İçerikleri

Bu kılavuzda referans verilen iki dosyanın **tam, değiştirilmemiş içeriği** — kopyalayıp doğrudan kullanabilirsiniz.

`esp32s3_config.h` — linorobot2_hardware/config/custom/

Bu projede fiilen kullanılan, doğrulanmış pin/kalibrasyon değerleriyle.

```
// ================================================================================
// LinoRobot2 Projesi — ESP32-S3 Donanım Yapılandırması
// ================================================================================
// Bu robotta fiilen kullanılan yapılandırma dosyasıdır. linorobot2_hardware
// projesinin ESP32-S3 config şablonu temel alınmış, aşağıdaki değerler bu
// robota özel olarak fiziksel ölçüm ve kalibrasyonla belirlenmiştir:
//
//   Şasi             : "Waffle" tarzı 4 katlı gövde
//   Motor            : 2x Pololu 25D, 99:1 dişli oranı, #3219 (48 CPR encoder)
//   Sürücü           : 2x BTS7960 (IBT_2)
//   MOTOR_MAX_RPM    : Pololu #3219 datasheet'inden (100 RPM, yüksüz @ 12V)
//   COUNTS_PER_REV1/2: gerçek 1m/2m mezura testiyle geriye hesaplanarak
//                      kalibre edildi (Motor1/SOL=2296, Motor2/SAĞ=2327) —
//                      yöntem için bkz. index.html §08
//   WHEEL_DIAMETER   : tur testiyle ölçülen etkin çap (64.6mm) — kumpasla
//                      ölçülen fiziksel 66mm'den teker kayması nedeniyle
//                      küçük bir fark var
//   LR_WHEELS_DISTANCE: sol-sağ tekerlek merkezleri arası, fiziksel ölçüm
//   Pin ataması      : Motor1(SOL)=GPIO12/13+encoder GPIO6/7,
//                      Motor2(SAĞ)=GPIO10/11+encoder GPIO4/5
//   WIFI_* / AGENT_IP/ SYSLOG_*: kullanılmıyor — ESP32↔RPi4 bağlantısı USB
//                      seri (micro-ROS, 921600 baud) üzerinden
//
// Aşağıdaki lisans bildirimi, temel alınan orijinal linorobot2_hardware
// şablonuna aittir (Apache 2.0 — değiştirilmeden korunmuştur).
// ================================================================================
//
// Copyright (c) 2021 Juan Miguel Jimeno
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

#ifndef ESP32S3_CONFIG_H
#define ESP32S3_CONFIG_H

#define LED_PIN LED_BUILTIN //used for debugging status

//uncomment the base you're building
#define LINO_BASE DIFFERENTIAL_DRIVE       // 2WD and Tracked robot w/ 2 motors
// #define LINO_BASE SKID_STEER            // 4WD robot
// #define LINO_BASE MECANUM               // Mecanum drive robot

//uncomment the motor driver you're using
// #define USE_GENERIC_2_IN_MOTOR_DRIVER      // Motor drivers with 2 Direction Pins(INA, INB) and 1 PWM(ENABLE) pin ie. L298, L293, VNH5019
// #define USE_GENERIC_1_IN_MOTOR_DRIVER   // Motor drivers with 1 Direction Pin(INA) and 1 PWM(ENABLE) pin.
#define USE_BTS7960_MOTOR_DRIVER        // BTS7970 Motor Driver using A4950 (<40V) module or DRV8833 (<10V)
// #define USE_ESC_MOTOR_DRIVER            // Motor ESC for brushless motors

//uncomment the IMU you're using
// #define USE_GY85_IMU
// #define USE_MPU6050_IMU
// #define USE_MPU9150_IMU
// #define USE_MPU9250_IMU
// #define USE_QMI8658_IMU
// #define USE_HMC5883L_MAG
// #define USE_AK8963_MAG
// #define USE_AK8975_MAG
// #define USE_AK09918_MAG
// #define USE_QMC5883L_MAG
// #define MAG_BIAS { 0, 0, 0 }
// #define IMU_TWEAK {}
// #define MAG_TWEAK {}

#define ACCEL_COV { 0.01, 0.01, 0.01 }
#define GYRO_COV { 0.001, 0.001, 0.001 }
#define ORI_COV { 0.01, 0.01, 0.01 }
#define MAG_COV { 1e-12, 1e-12, 1e-12 }
#define POSE_COV { 0.001, 0.001, 0.001, 0.001, 0.001, 0.001 }
#define TWIST_COV { 0.001, 0.001, 0.001, 0.003, 0.003, 0.003 }

#define K_P 0.6                            // P constant
#define K_I 0.8                            // I constant
#define K_D 0.5                            // D constant

/*
ROBOT ORIENTATION
         FRONT
    MOTOR1  MOTOR2  (2WD/ACKERMANN)
    MOTOR3  MOTOR4  (4WD/MECANUM)
         BACK
*/

//define your robot' specs here
#define MOTOR_MAX_RPM 100                  // motor's max RPM (Pololu #3219, 100 RPM no-load @ 12V)
```

```
#define MAX_RPM_RATIO 0.85                // max RPM allowed for each MAX_RPM_ALLOWED = MOTOR_MAX_RPM * MAX_RPM_RATIO
#define MOTOR_OPERATING_VOLTAGE 12        // motor's operating voltage (used to calculate max RPM)
#define MOTOR_POWER_MAX_VOLTAGE 12        // max voltage of the motor's power source (used to calculate max RPM)
#define MOTOR_POWER_MEASURED_VOLTAGE 12   // current voltage reading of the power connected to the motor (used for calibration)
#define COUNTS_PER_REV1 2296               // SOL motor — 2m/4.5cm sapma testinden geriye hesaplandı, bkz. index.html §08
#define COUNTS_PER_REV2 2327               // SAĞ motor — aynı yöntem
#define COUNTS_PER_REV3 450               // wheel3 encoder's no of ticks per rev
#define COUNTS_PER_REV4 450               // wheel4 encoder's no of ticks per rev
#define WHEEL_DIAMETER 0.0646             // wheel's diameter in meters — tur testiyle ölçülen etkin çap (kumpas: 0.066)
#define LR_WHEELS_DISTANCE 0.30           // distance between left and right wheels — fiziksel ölçüm
#define PWM_BITS 10                       // PWM Resolution of the microcontroller
#define PWM_FREQUENCY 20000               // PWM Frequency

// INVERT ENCODER COUNTS
#define MOTOR1_ENCODER_INV true
#define MOTOR2_ENCODER_INV false
#define MOTOR3_ENCODER_INV false
#define MOTOR4_ENCODER_INV false

// INVERT MOTOR DIRECTIONS
#define MOTOR1_INV true
#define MOTOR2_INV false
#define MOTOR3_INV false
#define MOTOR4_INV false

// ENCODER PINS
#define MOTOR1_ENCODER_A 6
#define MOTOR1_ENCODER_B 7

#define MOTOR2_ENCODER_A 4
#define MOTOR2_ENCODER_B 5

#define MOTOR3_ENCODER_A 39
#define MOTOR3_ENCODER_B 40

#define MOTOR4_ENCODER_A 41
#define MOTOR4_ENCODER_B 42

// MOTOR PINS
#ifdef USE_GENERIC_2_IN_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A 20
  #define MOTOR1_IN_B 1

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A 6
  #define MOTOR2_IN_B 8

  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A 23
  #define MOTOR3_IN_B 0

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A 3
  #define MOTOR4_IN_B 2

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif

#ifdef USE_GENERIC_1_IN_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A 20
  #define MOTOR1_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A 6
  #define MOTOR2_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A 23
  #define MOTOR3_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A 3
  #define MOTOR4_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif

#ifdef USE_BTS7960_MOTOR_DRIVER
  #define MOTOR1_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR1_IN_A 12
  #define MOTOR1_IN_B 13

  #define MOTOR2_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR2_IN_A 10
  #define MOTOR2_IN_B 11

  #define MOTOR3_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR3_IN_A 14
  #define MOTOR3_IN_B 15

  #define MOTOR4_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR4_IN_A 16
  #define MOTOR4_IN_B 17

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif

#ifdef USE_ESC_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR1_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR2_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder
```

```
  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR3_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR4_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define PWM_MAX 400
  #define PWM_MIN -PWM_MAX
#endif

#define AGENT_IP { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define AGENT_PORT 8888
#define WIFI_SSID "WIFI_SSID"
#define WIFI_PASSWORD "WIFI_PASSWORD"
// Enable WiFi with null terminated list of multiple APs SSID and password
// #define WIFI_AP_LIST {{"WIFI_SSID", "WIFI_PASSWORD"}, {NULL}}
#define WIFI_MONITOR 2 // min. period to send wifi signal strength to syslog
// #define USE_ARDUINO_OTA
// #define USE_SYSLOG
#define SYSLOG_SERVER { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define SYSLOG_PORT 514
#define DEVICE_HOSTNAME "esp32s3"
#define APP_NAME "hardware"
// #define USE_LIDAR_UDP
#define LIDAR_RXD 2
// #define LIDAR_PWM 15
#define LIDAR_SERIAL 1 // uart number
#define LIDAR_BAUDRATE 230400
#define LIDAR_SERVER { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define LIDAR_PORT 8889
#define BAUDRATE 921600
#define SDA_PIN 8 // specify I2C pins
#define SCL_PIN 9
#define NODE_NAME "esp32s3"
// #define TOPIC_PREFIX "esp32s3/"

// battery voltage ADC pin
#define BATTERY_PIN 1
// 3.3V ref, 12 bits ADC, 33k + 10k voltage divider
// #define USE_ADC_LUT
#ifdef USE_ADC_LUT
const int16_t ADC_LUT[4096] = { /* insert adc_calibrate data here */ };
#define BATTERY_ADJUST(v) (ADC_LUT[v] * (3.3 / 4096 * (33 + 10) / 10 * 1.0))
#else
#define BATTERY_ADJUST(v) ((v) * (3.3 / 4096 * (33 + 10) / 10))
#endif
// #define USE_INA219
// #define TRIG_PIN 31 // ultrasonic sensor HC-SR04
// #define ECHO_PIN 32
#define USE_SHORT_BRAKE // for shorter stopping distance
// #define WDT_TIMEOUT 60 // Sec
#define BOARD_INIT { \
    Wire.begin(SDA_PIN, SCL_PIN); \
    Wire.setClock(400000); \
}
// #define BOARD_INIT_LATE {}
// #define BOARD_LOOP {}

#ifdef USE_SYSLOG
#define RCCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){ \
    syslog(LOG_ERR, "%s RCCHECK failed %d", __FUNCTION__, temp_rc); \
    }}
#else
#define RCCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){ \
    flashLED(3); \
    }} // do not block
#endif

#endif
```

install_linorobot2.bash — RPi4 kurulum betiği, proje notuyla

Betik mantığı upstream'den değiştirilmemiştir; başına bu robotta nasıl çalıştırıldığını açıklayan bir proje notu eklenmiştir.

```
#!/usr/bin/env bash
# ================================================================================
# LinoRobot2 Projesi — Raspberry Pi 4 Kurulum Notu
# ================================================================================
# Bu robotta kurulu LIDAR: LDS-01 (ROBOTIS, hls_lfcd_lds_driver paketi, 230400 baud).
# Bu betik LDS-01'i desteklemez (LASER_SENSOR_ARRAY: ydlidar/xv11/ld06/ld19/
# stl27l/a1/a2/a3/c1/s1/s2/s3) — ikinci argüman olarak "-" verilip betiğin
# kendi LIDAR kurulumu atlanır, LDS-01 ayrıca elle eklenir (bkz. §07):
#
#     $ bash install_linorobot2.bash 2wd -
#
# Betiğin kendisi resmi linorobot2 deposundan (humble dalı) değiştirilmeden
# alınmıştır — aşağıdaki lisans/telif bildirimi orijinal yazara aittir.
# ================================================================================
#
# Copyright (c) 2021 Juan Miguel Jimeno
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http:#www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
```

```
set -e

ROSDISTRO="$(printenv ROS_DISTRO)"
BASE=$1
LASER_SENSOR=$2
DEPTH_SENSOR=$3
ARCH="$(uname -m)"
WORKSPACE="$HOME/linorobot2_ws"

ROBOT_TYPE_ARRAY=(2wd 4wd mecanum)
DEPTH_SENSOR_ARRAY=(realsense zed zedm zed2 zed2i oakd oakdlite oakdpro)
LASER_SENSOR_ARRAY=(ydlidar xv11 ld06 ld19 stl27l a1 a2 a3 c1 s1 s2 s3)
LASER_SENSOR_ARRAY+=(${DEPTH_SENSOR_ARRAY[@]})

if [ -z "$LASER_SENSOR" ]
    then
        LASER_SENSOR=""
fi

if [ -z "$DEPTH_SENSOR" ]
    then
        DEPTH_SENSOR=""
fi

function install_cuda_jetson {
    wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/sbsa/cuda-ubuntu2004.pin
    sudo mv cuda-ubuntu2004.pin /etc/apt/preferences.d/cuda-repository-pin-600
    wget http://developer.download.nvidia.com/compute/cuda/11.4.2/local_installers/cuda-repo-ubuntu2004-11-4-local_11.4.2-470.57.02-1_arm64.deb
    sudo dpkg -i cuda-repo-ubuntu2004-11-4-local_11.4.2-470.57.02-1_arm64.deb
    sudo apt-key add /var/cuda-repo-ubuntu2004-11-4-local/7fa2af80.pub #verify this
    sudo apt-get update
    sudo apt-get -y install cuda
    # Errors were encountered while processing:
    #  /tmp/apt-dpkg-install-TvUCLd/14-libnvidia-compute-470_470.57.02-0ubuntu1_arm64.deb
    #  /tmp/apt-dpkg-install-TvUCLd/18-libnvidia-gl-470_470.57.02-0ubuntu1_arm64.deb
    # E: Sub-process /usr/bin/dpkg returned an error code (1)
}

function install_xv11 {
    cd $WORKSPACE
    git clone https://github.com/mjstn/xv_11_driver src/xv_11_driver
    colcon build
    source $WORKSPACE/install/setup.bash
}

function install_ydlidar {
    cd /tmp
    git clone https://github.com/YDLIDAR/YDLidar-SDK.git
    mkdir YDLidar-SDK/build
    cd YDLidar-SDK/build
    cmake ..
    make
    sudo make install
    cd $WORKSPACE
    git clone https://github.com/YDLIDAR/ydlidar_ros2_driver src/ydlidar_ros2_driver
    chmod 0777 src/ydlidar_ros2_driver/startup/*
    sudo echo  'KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar.rules
    sudo echo  'KERNEL=="ttyACM*", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="5740", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar-V2.rules
    sudo echo  'KERNEL=="ttyUSB*", ATTRS{idVendor}=="067b", ATTRS{idProduct}=="2303", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar-2303.rules
    colcon build --symlink-install
    source $WORKSPACE/install/setup.bash
}

function install_ldlidar_stl_ros2 {
    cd $WORKSPACE
    git clone https://github.com/hippo5329/ldlidar_stl_ros2.git src/ldlidar_stl_ros2
    colcon build
    source $WORKSPACE/install/setup.bash
    cd /tmp
    wget https://raw.githubusercontent.com/linorobot/ldlidar/ros2/ldlidar.rules
    sudo cp ldlidar.rules /etc/udev/rules.d
}

function install_ld06 {
    install_ldlidar_stl_ros2
}

function install_ld19 {
    install_ldlidar_stl_ros2
}

function install_stl27l {
    install_ldlidar_stl_ros2
}

function install_sllidar_ros2 {
    cd $WORKSPACE
    git clone https://github.com/Slamtec/sllidar_ros2.git
    colcon build
    source $WORKSPACE/install/setup.bash
    sudo cp sllidar_ros2/scripts/rplidar.rules /etc/udev/rules.d
}

function install_a1 {
    install_sllidar_ros2
}

function install_a2 {
    install_sllidar_ros2
}

function install_a3 {
    install_sllidar_ros2
}

function install_c1 {
    install_sllidar_ros2
}

function install_s1 {
```

```
    install_sllidar_ros2
}

function install_s2 {
    install_sllidar_ros2
}

function install_s3 {
    install_sllidar_ros2
}

function install_realsense {
    sudo apt install -y ros-$ROS_DISTRO-realsense2-camera
    cd /tmp
    wget https://raw.githubusercontent.com/IntelRealSense/librealsense/master/config/99-realsense-libusb.rules
    sudo cp 99-realsense-libusb.rules /etc/udev/rules.d
}

function install_astra {
    cd $WORKSPACE
    sudo apt install -y libuvc-dev libopenni2-dev
    git clone https://github.com/linorobot/ros_astra_camera src/ros_astra_camera
    sudo cp src/ros_astra_camera/56-orbbec-usb.rules /etc/udev/rules.d/
    colcon build
    source $WORKSPACE/install/setup.bash
}

function install_zed {
    cd /tmp
    if [[ -f /etc/nv_tegra_release ]]
        then
            #TODO ADD CUDA INSTALLATION HERE
            wget https://download.stereolabs.com/zedsdk/3.5/jp45/jetsons -O zed_sdk
    elif lspci | grep VGA | grep -o NVIDIA
        then
            wget https://download.stereolabs.com/zedsdk/3.5/cu111/ubuntu20 -O zed_sdk
    else
        echo "Linux Machine not supported by Zed Camera"
        exit 1
    fi

    chmod +x zed_sdk
    ./zed_sdk -- silent
    cd $WORKSPACE

    git clone https://github.com/stereolabs/zed-ros2-wrapper src/zed-ros2-wrapper
    git clone https://github.com/ros-perception/image_common -b $ROS_DISTRO src/image_common #https://github.com/stereolabs/zed-ros2-wrapper#image-transport-and-topic-subscriptions
    rosdep install --from-paths src --ignore-src -r -y
    colcon build --symlink-install --cmake-args=-DCMAKE_BUILD_TYPE=Release
    # colcon build --symlink-install --cmake-args=-DCMAKE_BUILD_TYPE=Release --cmake-args=-DCUDA_TOOLKIT_ROOT_DIR=/usr/local/cuda-11.4
    source $WORKSPACE/install/setup.bash
    source ~/.bashrc
}

function install_zedm {
    install_zed
}

function install_zed2 {
    install_zed
}

function install_zed2i {
    install_zed
}

function install_oakd {
    echo 'SUBSYSTEM=="usb", ATTRS{idVendor}=="03e7", MODE="0666"' | sudo tee /etc/udev/rules.d/80-movidius.rules
    sudo udevadm control --reload-rules && sudo udevadm trigger
    sudo apt install ros-$ROS_DISTRO-depthai-ros
}

function install_oakdlite {
    install_oakd
}

function install_oakdpro {
    install_oakd
}

if [[ "$ROSDISTRO" == "" || "$ROSDISTRO" == "<unknown>" ]]
    then
        echo "No ROS2 distro detected"
        echo "Try running $ source /opt/ros/<ros_distro>/setup.bash and try again."
        exit 1
fi

if [ "$*" == "" ]
    then
        echo "No arguments provided"
        echo
        echo "Example: $ bash install_linorobot2.bash 2wd a1"
        echo "Example: $ bash install_linorobot2.bash 2wd a1 realsense"
        echo "Example: $ bash install_linorobot2.bash 2wd - realsense"
        echo "Example: $ bash install_linorobot2.bash 2wd"

        echo
        exit 1
fi

if [[ "$BASE" != "ci" ]] && !(printf '%s\n' "${ROBOT_TYPE_ARRAY[@]}" | grep -xq $BASE)
    then
        echo "Invalid linorobot base: $1"
        echo
        echo "Valid Options:"
        for key in "${!ROBOT_TYPE_ARRAY[@]}"; do echo "${ROBOT_TYPE_ARRAY[$key]}"; done
        echo
        exit 1
fi

if [[ "$BASE" != "ci" && "$LASER_SENSOR" != "" && "$LASER_SENSOR" != "-" ]] && !(printf '%s\n' "${LASER_SENSOR_ARRAY[@]}" | grep -xq $LASER_SENSOR)
```

```
    then
        echo "Invalid linorobot2 laser sensor: $LASER_SENSOR"
        echo
        echo "Valid Options:"
        for key in "${!LASER_SENSOR_ARRAY[@]}"; do echo "${LASER_SENSOR_ARRAY[$key]}"; done
        echo
        exit 1
fi

if [[ "$BASE" != "ci" && "$DEPTH_SENSOR" != "" ]] && !(printf '%s\n' "${DEPTH_SENSOR_ARRAY[@]}" | grep -xq $DEPTH_SENSOR)
    then
        echo "Invalid linorobot2 depth sensor: $DEPTH_SENSOR"
        echo
        echo "Valid Options:"
        for key in "${!DEPTH_SENSOR_ARRAY[@]}"; do echo "${DEPTH_SENSOR_ARRAY[$key]}"; done
        echo
        exit 1
fi

if [[ "$BASE" != "ci" ]]
    then
        echo
        echo "You are installing linorobot2 on your robot computer."
        echo
        echo "==========SUMMARY==========="
        echo "ROBOT TYPE   : $BASE"
        echo "LASER SENSOR : $LASER_SENSOR"
        echo "DEPTH SENSOR : $DEPTH_SENSOR"
        echo
        echo "This installer will edit your ~/.bashrc."
        echo "Create a linorobot2_ws on your $HOME directory."
        echo "Install linorobot2 ROS2 dependencies."
        echo "Install udev rules on /etc/udev/rules.d folder."
        echo -n "Enter [y] to continue. "
        read reply
        if [[ "$reply" != "y" && "$reply" != "Y" ]]
            then
                echo "Exiting now."
                exit 1
        fi
fi

echo
echo "INSTALLING NOW...."
echo

#### 1.1 Source your ROS2 distro and workspace
cd $HOME
mkdir -p $WORKSPACE/src
source /opt/ros/$ROS_DISTRO/setup.bash
cd $WORKSPACE
colcon build
source $WORKSPACE/install/setup.bash

#### 1.2/1.3 Install LIDAR/Depth Sensor ROS2 drivers:
if (printf '%s\n' "${LASER_SENSOR_ARRAY[@]}" | grep -xq $LASER_SENSOR)
    then
        install_$LASER_SENSOR
fi

if (printf '%s\n' "${DEPTH_SENSOR_ARRAY[@]}" | grep -xq $DEPTH_SENSOR)
    then
        install_$DEPTH_SENSOR
fi

if [[ "$BASE" == "ci" ]]
    then
        for key in "${!LASER_SENSOR_ARRAY[@]}"; do  install_${LASER_SENSOR_ARRAY[$key]}; done
fi

#### 1.4 Download and install micro-ROS:
cd $WORKSPACE
git clone -b $ROS_DISTRO https://github.com/micro-ROS/micro_ros_setup.git src/micro_ros_setup
sudo apt install -y python3-vcstool build-essential
sudo apt update && rosdep update
rosdep install --from-path src --ignore-src -y
colcon build
source $WORKSPACE/install/setup.bash

#### 1.5 Setup micro-ROS agent:
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
source $WORKSPACE/install/setup.bash

#### 2.1 Download linorobot2:
cd $WORKSPACE
git clone -b $ROS_DISTRO https://github.com/linorobot/linorobot2 src/linorobot2

#### 2.2 Ignore Gazebo Packages on robot computer (optional)
cd $WORKSPACE/src/linorobot2/linorobot2_gazebo
touch COLCON_IGNORE

#### 2.3 Install linorobot2 package:
cd $WORKSPACE
rosdep update && rosdep install --from-path src --ignore-src -y --skip-keys microxrcedds_agent
colcon build
source $WORKSPACE/install/setup.bash

## ENV Variables
if [[ "$BASE" != "ci" ]]
    then
        ### 1. Robot Type
        echo "export LINOROBOT2_BASE=$BASE" >> ~/.bashrc
        ### 2. Sensors
        if [[ "$LASER_SENSOR" != "-" ||  "$LASER_SENSOR" != "" ]]
            then
                echo "export LINOROBOT2_LASER_SENSOR=$LASER_SENSOR" >> ~/.bashrc
        fi

        if [[ "$DEPTH_SENSOR" != "-" ||  "$DEPTH_SENSOR" != "" ]]
            then
```

```
            echo "export LINOROBOT2_DEPTH_SENSOR=$DEPTH_SENSOR" >> ~/.bashrc
        fi
        echo
        echo "Do you want to add sourcing of linorobot2_ws on your ~/.bashrc?"
        echo -n "Yes [y] or No [n]: "
        read reply
        if [[ "$reply" == "y" || "$reply" == "Y" ]]
            then
                echo "source \$HOME/linorobot2_ws/install/setup.bash" >> ~/.bashrc
        else
            echo
            echo "Remember to run $ source ~/linorobot2_ws/install/setup.bash every time you open a terminal."
        fi
fi

echo
echo "INSTALLATION DONE."
echo
echo "Restart your robot computer now."
```

`2wd_properties.urdf.xacro` — linorobot2_description/urdf/

Bu robotun gerçek mezura ölçüleriyle güncellenmiş URDF özellik dosyası.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://ros.org/wiki/xacro">
  <!-- Degerler gercek mezura olcumleriyle belirlenmistir (Waffle sasi) -->
  <xacro:property name="base_length" value="0.27" />
  <xacro:property name="base_width" value="0.27" />
  <xacro:property name="base_height" value="0.003" />
  <xacro:property name="base_mass" value="1" />

  <xacro:property name="wheel_radius" value="0.0325" /> <!-- 6.5cm cap olcumu -->
  <xacro:property name="wheel_width" value="0.017" />
  <xacro:property name="wheel_pos_x" value="0.085" /> <!-- ana teker on kenardan 5cm, merkeze gore +8.5cm -->
  <xacro:property name="wheel_pos_y" value="0.1565" /> <!-- teker disi 33cm - 1.7cm genislik = 31.3cm merkez-merkez -->
  <xacro:property name="wheel_pos_z" value="0.023" />
  <xacro:property name="wheel_mass" value=".1" />
  <xacro:property name="wheel_torque" value="20" />
  <xacro:property name="front_caster_wheel" value="false" /> <!-- onde caster yok, ikisi de arkada -->
  <xacro:property name="rear_caster_wheel" value="true" />

  <xacro:property name="laser_pose">
    <origin xyz="0.035 0 0.15" rpy="0 0 0"/> <!-- on kenardan 10cm, ortali, yerden 15cm -->
  </xacro:property>

  <xacro:property name="depth_sensor_pose">
    <origin xyz="0.14 0.0 0.045" rpy="0 0 0"/>
  </xacro:property>

  <xacro:property name="rear_camera_sensor_pose">
    <origin xyz="-0.14 0.0 0.045" rpy="0 0 3.14159"/>
  </xacro:property>

</robot>
```

`linorobot2_bringup.service` — /etc/systemd/system/

Temel sistem: micro-ROS agent + EKF + robot_state_publisher.

```
[Unit]
Description=linorobot2 temel sistemi (micro-ROS agent + EKF + robot tanimi)
After=network.target

[Service]
Type=simple
User=pi4
Group=pi4
Environment=HOME=/home/pi4
Environment=LINOROBOT2_BASE=2wd
ExecStart=/bin/bash -lc 'unset LINOROBOT2_LASER_SENSOR; source /opt/ros/humble/setup.bash && source /home/pi4/linorobot2_ws/install/setup.bash && exec ros2 launch linorobot2_bringup bringup.launch.py micro_ros_baud
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

`joy_node.service` — /etc/systemd/system/

F710 kumandasını okuyup /joy yayınlar.

```
[Unit]
Description=Logitech F710 joystick reader (joy_node)
After=network.target

[Service]
Type=simple
User=pi4
Group=pi4
Environment=HOME=/home/pi4
ExecStart=/bin/bash -lc 'source /opt/ros/humble/setup.bash && exec ros2 run joy joy_node'
Restart=always
RestartSec=3
```

```
[Install]
WantedBy=multi-user.target
```

`joystick_teleop.service` — /etc/systemd/system/

dpad_teleop.py'yi çalıştırır — /joy'u /cmd_vel'e çevirir.

```
[Unit]
Description=D-pad + sag cubuk teleop (dpad_teleop.py -> /cmd_vel)
After=network.target joy_node.service linorobot2_bringup.service
Wants=joy_node.service

[Service]
Type=simple
User=pi4
Group=pi4
Environment=HOME=/home/pi4
ExecStart=/bin/bash -lc 'source /opt/ros/humble/setup.bash && source /home/pi4/linorobot2_ws/install/setup.bash && exec python3 /home/pi4/joystick_ws/dpad_teleop.py'
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

`start_robot.sh` — ~/ (RPi4)

LIDAR + SLAM/Nav2'yi başlatan, durunca köprüyü otomatik tazeleyen script.

```
#!/bin/bash

LOG_DIR="$HOME/robot_logs"
mkdir -p "$LOG_DIR"

cleanup() {
  echo ""
  echo "Kapatiliyor (LIDAR + SLAM)..."
  kill $(jobs -p) 2>/dev/null
  wait 2>/dev/null
  echo "Temel sistem (micro-ROS koprusu) temiz durum icin yeniden baslatiliyor..."
  sudo systemctl restart linorobot2_bringup.service
  sleep 3
  echo "Tamam. Koprunu tekrar ayakta, joystick/cmd_vel calismali."
}
trap cleanup EXIT INT TERM

source /opt/ros/humble/setup.bash
source "$HOME/linorobot2_ws/install/setup.bash"

echo "[0/2] Temel sistemin (linorobot2_bringup.service) hazir olmasi bekleniyor..."
if ! systemctl is-active --quiet linorobot2_bringup.service; then
  echo "  Servis aktif degil, baslatiliyor..."
  sudo systemctl start linorobot2_bringup.service
fi

for i in $(seq 1 20); do
  if ros2 topic list 2>/dev/null | grep -q "^/odom/unfiltered$"; then
    echo "  Temel sistem hazir (/odom/unfiltered goruluyor)."
    break
  fi
  sleep 1
  if [ "$i" -eq 20 ]; then
    echo "  UYARI: 20 saniyede /odom/unfiltered gorulemedi, yine de devam ediliyor."
    echo "  Kontrol icin: systemctl status linorobot2_bringup.service"
  fi
done

echo "[1/2] LIDAR surucusu baslatiliyor (LDS-01 / hls_lfcd_lds_driver)..."
ros2 launch hls_lfcd_lds_driver hlds_laser.launch.py \
  > "$LOG_DIR/lidar.log" 2>&1 &
sleep 3

echo "[2/2] SLAM + Nav2 baslatiliyor..."
ros2 launch linorobot2_navigation slam.launch.py \
  > "$LOG_DIR/slam.log" 2>&1 &

echo ""
echo "Hepsi baslatildi. Loglar: $LOG_DIR/ (lidar.log, slam.log)"
echo "Temel sistemin loglari icin: journalctl -u linorobot2_bringup.service -f"
echo "Durdurmak icin Ctrl+C'ye bas."
echo ""

wait
```

`dpad_teleop.py` — ~/joystick_ws/ (RPi4)

D-pad + sağ çubuk → /cmd_vel özel teleop düğümü.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Joy
from geometry_msgs.msg import Twist

LINEAR_SPEED = 0.29
ANGULAR_SCALE = 0.75
```

```
DEADMAN_BUTTON = 4
DPAD_DOWN = 1
DPAD_UP = 3
RIGHT_STICK_X_AXIS = 4

class DpadTeleop(Node):
    def __init__(self):
        super().__init__('dpad_teleop')
        self.create_subscription(Joy, 'joy', self.joy_cb, 10)
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)

    def joy_cb(self, msg: Joy):
        twist = Twist()
        if msg.buttons[DEADMAN_BUTTON]:
            if msg.buttons[DPAD_UP]:
                twist.linear.x = LINEAR_SPEED
            elif msg.buttons[DPAD_DOWN]:
                twist.linear.x = -LINEAR_SPEED
            twist.angular.z = msg.axes[RIGHT_STICK_X_AXIS] * ANGULAR_SCALE
        self.pub.publish(twist)

def main():
    rclpy.init()
    node = DpadTeleop()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

go_1m_test.py — ~/ (RPi4)

Odometri kalibrasyonu için otomatik 1m/2m düz sürüş test betiği.

```
#!/usr/bin/env python3
import math
import sys
import time

import rclpy
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from rclpy.node import Node

TARGET_M = 1.0
CRUISE_MPS = 0.10
CREEP_MPS = 0.05
CREEP_ZONE_M = 0.15
TIMEOUT_S = 30.0


class OneMeterTest(Node):
    def __init__(self):
        super().__init__("one_meter_test")
        self.pub = self.create_publisher(Twist, "/cmd_vel", 10)
        self.create_subscription(Odometry, "/odom/unfiltered", self.odom_cb, 10)
        self.x0 = None
        self.x = self.y = None
        self.start_t = None
        self.done = False
        self.timer = self.create_timer(0.05, self.step)

    def odom_cb(self, msg):
        self.x = msg.pose.pose.position.x
        self.y = msg.pose.pose.position.y
        if self.x0 is None:
            self.x0, self.y0 = self.x, self.y

    def distance(self):
        return math.hypot(self.x - self.x0, self.y - self.y0)

    def send(self, v):
        t = Twist()
        t.linear.x = v
        self.pub.publish(t)

    def stop(self):
        for _ in range(20):
            self.send(0.0)
            time.sleep(0.02)

    def step(self):
        if self.done:
            return
        if self.x0 is None:
            if self.start_t is None:
                self.start_t = time.time()
            if time.time() - self.start_t > 5.0:
                print("HATA: 5 sn icinde /odom/unfiltered gelmedi, durduruluyor.")
                self.finish()
            return
        if self.start_t is None:
            self.start_t = time.time()
            print("Baslangic alindi, ileri gidiliyor...")
        d = self.distance()
        if d >= TARGET_M:
            self.finish(d)
            return
        if time.time() - self.start_t > TIMEOUT_S:
            print(f"ZAMAN ASIMI ({TIMEOUT_S}s), durduruluyor. Odometre mesafesi: {d:.3f} m")
            self.finish(d)
            return
        v = CREEP_MPS if (TARGET_M - d) < CREEP_ZONE_M else CRUISE_MPS
        self.send(v)
```

```
    def finish(self, d=None):
        self.done = True
        self.timer.cancel()
        self.stop()
        if d is None and self.x0 is not None:
            d = self.distance()
        if d is not None:
            print("")
            print(f"Odometre (Euclid) mesafe : {d:.3f} m")
            print(f"Hedef                    : {TARGET_M:.3f} m")
            print("Simdi mezuru ile gercek mesafeyi olc ve buraya yaz.")
        print(f"Son odom konumu  x={self.x} y={self.y}" if self.x is not None else "")


def main():
    rclpy.init()
    node = OneMeterTest()
    try:
        while rclpy.ok() and not node.done:
            rclpy.spin_once(node, timeout_sec=0.05)
    except KeyboardInterrupt:
        print("\nCtrl+C: durduruluyor.")
        node.stop()
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    sys.exit(main())
```

ekf.yaml — linorobot2_base/config/ (upstream yoldaki hali)

EKF güncelleme frekansı stoktan 50.0'dan 10.0'a düşürüldü (bkz. §13), geri kalanı stok.

```
ekf_filter_node:
    ros__parameters:
        frequency: 10.0
        two_d_mode: true
        publish_tf: true

        map_frame: map
        odom_frame: odom
        base_link_frame: base_footprint
        world_frame: odom

        #x     , y     , z,
        #roll  , pitch , yaw,
        #vx    , vy    , vz,
        #vroll , vpitch, vyaw,
        #ax    , ay    , az
        odom0: odom/unfiltered
        odom0_config: [false, false, false,
                       false, false, false,
                       true, true, false,
                       false, false, true,
                       false, false, false]

        imu0: imu/data
        imu0_config: [false, false, false,
                      false, false, false,
                      false, false, false,
                      false, false, true,
                      false, false, false]
```

navigation.yaml — linorobot2_navigation/config/ (upstream yoldaki hali)

Nav2 parametreleri — robotun gerçek azami hızı (0.29 m/s) ve gövde yarıçapına (0.13m) göre güncellenen 4 parametre dışında stok (bkz. §13).

```
amcl:
  ros__parameters:
    use_sim_time: false
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: "base_footprint"
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: "map"
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 100.0
    laser_min_range: -1.0
    laser_model_type: "likelihood_field"
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: "odom"
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
```

```
    save_pose_rate: 0.5
    sigma_hit: 0.2
    tf_broadcast: true
    transform_tolerance: 1.0
    update_min_a: 0.2
    update_min_d: 0.25
    z_hit: 0.5
    z_max: 0.05
    z_rand: 0.5
    z_short: 0.05
    scan_topic: scan

amcl_map_client:
  ros__parameters:
    use_sim_time: false

amcl_rclcpp_node:
  ros__parameters:
    use_sim_time: false

bt_navigator:
  ros__parameters:
    use_sim_time: false
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    enable_groot_monitoring: true
    groot_zmq_publisher_port: 1666
    groot_zmq_server_port: 1667
    plugin_lib_names:
    - nav2_compute_path_to_pose_action_bt_node
    - nav2_compute_path_through_poses_action_bt_node
    - nav2_follow_path_action_bt_node
    - nav2_back_up_action_bt_node
    - nav2_spin_action_bt_node
    - nav2_wait_action_bt_node
    - nav2_clear_costmap_service_bt_node
    - nav2_is_stuck_condition_bt_node
    - nav2_goal_reached_condition_bt_node
    - nav2_goal_updated_condition_bt_node
    - nav2_initial_pose_received_condition_bt_node
    - nav2_reinitialize_global_localization_service_bt_node
    - nav2_rate_controller_bt_node
    - nav2_distance_controller_bt_node
    - nav2_speed_controller_bt_node
    - nav2_truncate_path_action_bt_node
    - nav2_goal_updater_node_bt_node
    - nav2_recovery_node_bt_node
    - nav2_pipeline_sequence_bt_node
    - nav2_round_robin_node_bt_node
    - nav2_transform_available_condition_bt_node
    - nav2_time_expired_condition_bt_node
    - nav2_distance_traveled_condition_bt_node
    - nav2_single_trigger_bt_node
    - nav2_is_battery_low_condition_bt_node
    - nav2_navigate_through_poses_action_bt_node
    - nav2_navigate_to_pose_action_bt_node
    - nav2_remove_passed_goals_action_bt_node
    - nav2_planner_selector_bt_node
    - nav2_controller_selector_bt_node
    - nav2_goal_checker_selector_bt_node

bt_navigator_rclcpp_node:
  ros__parameters:
    use_sim_time: false

controller_server:
  ros__parameters:
    use_sim_time: false
    controller_frequency: 20.0
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
    min_theta_velocity_threshold: 0.001
    failure_tolerance: 0.3
    progress_checker_plugin: "progress_checker"
    goal_checker_plugins: ["general_goal_checker"] # "precise_goal_checker"
    controller_plugins: ["FollowPath"]

    # Progress checker parameters
    progress_checker:
      plugin: "nav2_controller::SimpleProgressChecker"
      required_movement_radius: 0.5
      movement_time_allowance: 10.0

    # Goal checker parameters
    general_goal_checker:
      stateful: true
      plugin: "nav2_controller::SimpleGoalChecker"
      xy_goal_tolerance: 0.25
      yaw_goal_tolerance: 0.25
    # DWB parameters
    FollowPath:
      plugin: "nav2_rotation_shim_controller::RotationShimController"
      angular_dist_threshold: 0.785
      forward_sampling_distance: 0.5
      rotate_to_heading_angular_vel: 1.8
      max_angular_accel: 3.2
      simulate_ahead_time: 1.0
      plugin: "dwb_core::DWBLocalPlanner"
      debug_trajectory_details: true
      min_vel_x: 0.0
      min_vel_y: 0.0
      max_vel_x: 0.29
      max_vel_y: 0.0
      max_vel_theta: 0.75
      min_speed_xy: 0.0
      max_speed_xy: 0.29
      min_speed_theta: 0.0
      # Add high threshold velocity for turtlebot 3 issue.
```

```
      # https://github.com/ROBOTIS-GIT/turtlebot3_simulations/issues/75
      acc_lim_x: 2.5
      acc_lim_y: 0.0
      acc_lim_theta: 3.2
      decel_lim_x: -2.5
      decel_lim_y: 0.0
      decel_lim_theta: -3.2
      vx_samples: 20
      vy_samples: 5
      vtheta_samples: 20
      sim_time: 1.7
      linear_granularity: 0.05
      angular_granularity: 0.025
      transform_tolerance: 0.2
      xy_goal_tolerance: 0.25
      trans_stopped_velocity: 0.25
      short_circuit_trajectory_evaluation: true
      stateful: true
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
      BaseObstacle.scale: 0.02
      PathAlign.scale: 32.0
      PathAlign.forward_point_distance: 0.1
      GoalAlign.scale: 24.0
      GoalAlign.forward_point_distance: 0.1
      PathDist.scale: 32.0
      GoalDist.scale: 24.0
      RotateToGoal.scale: 32.0
      RotateToGoal.slowing_factor: 5.0
      RotateToGoal.lookahead_time: -1.0

controller_server_rclcpp_node:
  ros__parameters:
    use_sim_time: false

global_costmap:
  global_costmap:
    ros__parameters:
      use_sim_time: false
      footprint_padding: 0.03
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      robot_radius: 0.13 # eskisi 0.10 (20cm genislik) idi, Waffle sasiye gecis sonrasi 26cm genislige gore guncellendi
      resolution: 0.05
      plugins: ["static_layer", "obstacle_layer", "voxel_layer", "inflation_layer"]
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
        observation_sources: scan base_scan
        footprint_clearing_enabled: true
        max_obstacle_height: 2.0
        combination_method: 1
        scan:
          topic: /scan
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          max_obstacle_height: 2.0
          min_obstacle_height: 0.0
          clearing: true
          marking: true
          data_type: "LaserScan"
          inf_is_valid: false
        base_scan:
          topic: /base/scan
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          max_obstacle_height: 2.0
          min_obstacle_height: 0.0
          clearing: true
          marking: true
          data_type: "LaserScan"
          inf_is_valid: false
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        enabled: true
        footprint_clearing_enabled: true
        max_obstacle_height: 2.0
        publish_voxel_map: true
        origin_z: 0.0
        z_resolution: 0.05
        z_voxels: 16
        max_obstacle_height: 2.0
        unknown_threshold: 15
        mark_threshold: 0
        observation_sources: pointcloud
        combination_method: 1
        pointcloud:  # no frame set, uses frame from message
          topic: /camera/depth/color/points
          max_obstacle_height: 2.0
          min_obstacle_height: 0.01
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          clearing: true
          marking: true
          data_type: "PointCloud2"
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: true
        enabled: true
        subscribe_to_updates: true
        transform_tolerance: 0.1
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        enabled: true
```

```
        inflation_radius: 0.55
        cost_scaling_factor: 1.0
        inflate_unknown: false
        inflate_around_unknown: true
      always_send_full_costmap: true

local_costmap:
  local_costmap:
    ros__parameters:
      use_sim_time: false
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05
      robot_radius: 0.13 # eskisi 0.10 (20cm genislik) idi, Waffle sasiye gecis sonrasi 26cm genislige gore guncellendi
      resolution: 0.05
      plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
        observation_sources: scan base_scan
        footprint_clearing_enabled: true
        max_obstacle_height: 2.0
        combination_method: 1
        scan:
          topic: /scan
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          max_obstacle_height: 2.0
          min_obstacle_height: 0.0
          clearing: false
          marking: true
          data_type: "LaserScan"
          inf_is_valid: false
        base_scan:
          topic: /base/scan
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          max_obstacle_height: 2.0
          min_obstacle_height: 0.0
          clearing: false
          marking: true
          data_type: "LaserScan"
          inf_is_valid: false
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        enabled: true
        footprint_clearing_enabled: true
        max_obstacle_height: 2.0
        publish_voxel_map: true
        origin_z: 0.0
        z_resolution: 0.05
        z_voxels: 16
        max_obstacle_height: 2.0
        unknown_threshold: 15
        mark_threshold: 0
        observation_sources: pointcloud
        combination_method: 1
        pointcloud:  # no frame set, uses frame from message
          topic: /camera/depth/color/points
          max_obstacle_height: 2.0
          min_obstacle_height: 0.01
          obstacle_max_range: 2.5
          obstacle_min_range: 0.0
          raytrace_max_range: 3.0
          raytrace_min_range: 0.0
          clearing: true
          marking: true
          data_type: "PointCloud2"
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        enabled: true
        inflation_radius: 0.55
        cost_scaling_factor: 1.0
        inflate_unknown: false
        inflate_around_unknown: true

map_server:
  ros__parameters:
    use_sim_time: false
    yaml_filename: "turtlebot3_world.yaml"

map_saver:
  ros__parameters:
    use_sim_time: false
    save_map_timeout: 5.0
    free_thresh_default: 0.25
    occupied_thresh_default: 0.65
    map_subscribe_transient_local: true

planner_server:
  ros__parameters:
    expected_planner_frequency: 20.0
    use_sim_time: false
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true

planner_server_rclcpp_node:
  ros__parameters:
```

```
    use_sim_time: false

recoveries_server:
  ros__parameters:
    costmap_topic: local_costmap/costmap_raw
    footprint_topic: local_costmap/published_footprint
    cycle_frequency: 10.0
    recovery_plugins: ["spin", "backup", "wait"]
    spin:
      plugin: "nav2_recoveries/Spin"
    backup:
      plugin: "nav2_recoveries/BackUp"
    wait:
      plugin: "nav2_recoveries/Wait"
    global_frame: odom
    robot_base_frame: base_link
    transform_timeout: 0.1
    use_sim_time: false
    simulate_ahead_time: 2.0
    max_rotational_vel: 1.0
    min_rotational_vel: 0.4
    rotational_acc_lim: 3.2

robot_state_publisher:
  ros__parameters:
    use_sim_time: false

waypoint_follower:
  ros__parameters:
    loop_rate: 20
    stop_on_failure: false
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: true
      waypoint_pause_duration: 200

velocity_smoother:
  ros__parameters:
    smoothing_frequency: 20.0
    scale_velocities: False
    feedback: "OPEN_LOOP"
    max_velocity: [0.29, 0.0, 2.5]
    min_velocity: [-0.29, 0.0, -2.5]
    max_accel: [2.5, 0.0, 3.2]
    max_decel: [-2.5, 0.0, -3.2]
    odom_topic: "odom"
    odom_duration: 0.1
    deadband_velocity: [0.0, 0.0, 0.0]
    velocity_timeout: 1.0
```

install_agent_service.sh — ~/ (RPi4)

linorobot2_bringup.service'i yazan, etkinleştiren ve başlatan kurulum betiği — ExecStart içinde LINOROBOT2_LASER_SENSOR'ü bilerek unset eder (bkz. §07).

```
#!/usr/bin/env bash
set -e

ORBIROB_USER="${USER}"
USER_HOME="${HOME}"

BRINGUP_SERVICE="/etc/systemd/system/linorobot2_bringup.service"
ROS_SETUP="/opt/ros/humble/setup.bash"
LINOROBOT2_SETUP="${USER_HOME}/linorobot2_ws/install/setup.bash"

echo "================================================"
echo " linorobot2_bringup.service kurulumu"
echo "================================================"
echo "Kullanici: ${ORBIROB_USER}"
echo "Ev dizini: ${USER_HOME}"
echo

if [ ! -f "${ROS_SETUP}" ]; then
    echo "HATA: ROS2 Humble bulunamadi: ${ROS_SETUP}"
    exit 1
fi

if [ ! -f "${LINOROBOT2_SETUP}" ]; then
    echo "HATA: linorobot2_ws derlenmemis: ${LINOROBOT2_SETUP}"
    exit 1
fi

echo "Servis dosyasi yaziliyor: ${BRINGUP_SERVICE}"

sudo tee "${BRINGUP_SERVICE}" >/dev/null <<EOF
[Unit]
Description=linorobot2 temel sistemi (micro-ROS agent + EKF + robot tanimi)
After=network.target

[Service]
Type=simple
User=${ORBIROB_USER}
Group=${ORBIROB_USER}
Environment=HOME=${USER_HOME}
ExecStart=/bin/bash -lc 'unset LINOROBOT2_LASER_SENSOR; source ${ROS_SETUP} && source ${LINOROBOT2_SETUP} && exec ros2 launch linorobot2_bringup bringup.launch.py micro_ros_baudrate:=921600'
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF

echo "systemd yeniden yukleniyor..."
sudo systemctl daemon-reload
```

```
echo "linorobot2_bringup.service etkinlestiriliyor (acilista otomatik baslasin diye)..."
sudo systemctl enable linorobot2_bringup.service

echo "Servis simdi baslatiliyor..."
sudo systemctl restart linorobot2_bringup.service

sleep 3

echo
echo "================================================"
echo " Kurulum tamamlandi"
echo "================================================"
echo -n "Durum: "
systemctl is-enabled linorobot2_bringup.service
echo
echo "Detayli durumu gormek icin:"
echo "  systemctl status linorobot2_bringup.service"
echo "Canli loglari izlemek icin:"
echo "  journalctl -u linorobot2_bringup.service -f"
```

baud_scan.py — ~/ (RPi4)

LIDAR için doğru baud hızını A5/5A sync baytına bakarak tarayan teşhis aracı (RPLIDAR döneminden kalma, tarihsel referans).

```
#!/usr/bin/env python3
"""RPLIDAR icin dogru baud'u bulmak icin taranan hizlarda a5 5a sync baytini arar."""
import serial
import time

PORT = "/dev/ttyUSB0"
BAUDS = [9600, 19200, 57600, 115200, 128000, 230400, 256000, 460800, 921600, 1000000]

for baud in BAUDS:
    try:
        ser = serial.Serial(PORT, baud, timeout=1.5)
        ser.reset_input_buffer()
        time.sleep(0.3)
        data = ser.read(500)
        ser.close()
        has_sync = b'\xa5\x5a' in data
        nonzero = len(data) - data.count(0)
        print(f"baud={baud:8d}  bayt_sayisi={len(data):4d}  a5-5a_sync={'EVET!' if has_sync else 'hayir':6s}  ornek={data[:16].hex()}")
    except Exception as e:
        print(f"baud={baud:8d}  HATA: {e}")
```

joint_track.py — ~/ (RPi4)

/joint_states üzerinden sol/sağ teker devrini (tur cinsinden) izler — WHEEL_DIAMETER tur testinde kullanılan araç (bkz. §08).

```
import math, rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class Track(Node):
    def __init__(self):
        super().__init__("joint_track")
        self.create_subscription(JointState, "/joint_states", self.cb, 10)
        self.start = None; self.last = None
    def cb(self, m):
        pos = dict(zip(m.name, m.position))
        if not all(k in pos for k in ("left_wheel_joint", "right_wheel_joint")): return
        self.last = pos
        if self.start is None: self.start = pos
    def report(self):
        if not self.start or not self.last:
            print("joint verisi yok"); return
        for k in ("left_wheel_joint", "right_wheel_joint"):
            print(f"{k}: {(self.last[k]-self.start[k])/(2*math.pi):.3f} tur")

rclpy.init(); n = Track()
try:
    rclpy.spin(n)
except KeyboardInterrupt:
    n.report()
```

odom_track.py — ~/ (RPi4)

/odom/unfiltered üzerinden kat edilen yolu ve başlangıca uzaklığı canlı izler — sapma/kalibrasyon testlerinde kullanılan araç (bkz. §08).

```
import math, time, rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry

class Track(Node):
    def __init__(self):
        super().__init__("odom_track")
        self.create_subscription(Odometry, "/odom/unfiltered", self.cb, 10)
        self.x0 = None; self.last = None; self.path = 0.0; self.t = time.time()
    def cb(self, m):
```

```
        x, y = m.pose.pose.position.x, m.pose.pose.position.y
        if self.x0 is None:
            self.x0, self.y0 = x, y; self.last = (x, y); return
        self.path += math.hypot(x - self.last[0], y - self.last[1]); self.last = (x, y)
        if time.time() - self.t > 0.5:
            self.t = time.time()
            print(f"baslangica uzaklik: {math.hypot(x-self.x0, y-self.y0):.3f} m   toplam yol: {self.path:.3f} m", flush=True)

rclpy.init(); n = Track()
try:
    rclpy.spin(n)
except KeyboardInterrupt:
    print(f"\nSON -> baslangica uzaklik: {math.hypot(n.last[0]-n.x0, n.last[1]-n.y0):.3f} m   toplam yol: {n.path:.3f} m")
```